

Rocket 食用指南

风险告知

本程序基于第三方工具进一步开发，无法保证底层代码的稳定性和安全性。使用过程中难免会出现软件或者硬件层面的问题，由此产生的盈亏，使用者需要自行承担。

一、 实盘前的准备

1、打开手机 APP “翼启航”，依次点击：我的 - 业务办理 - 委托方式修改 - 勾选 QMT 量化。

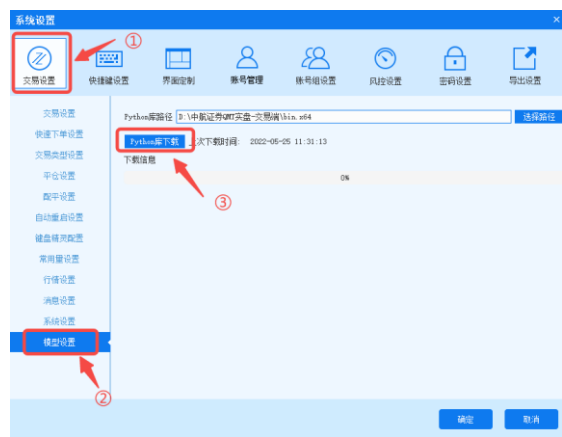
a) 注意：此步骤仅中航证券需要，其他券商不需要。



2、登录大 QMT，依次点击：设置 - 账号管理 - 添加 - 输入客户号&密码 - 确定。

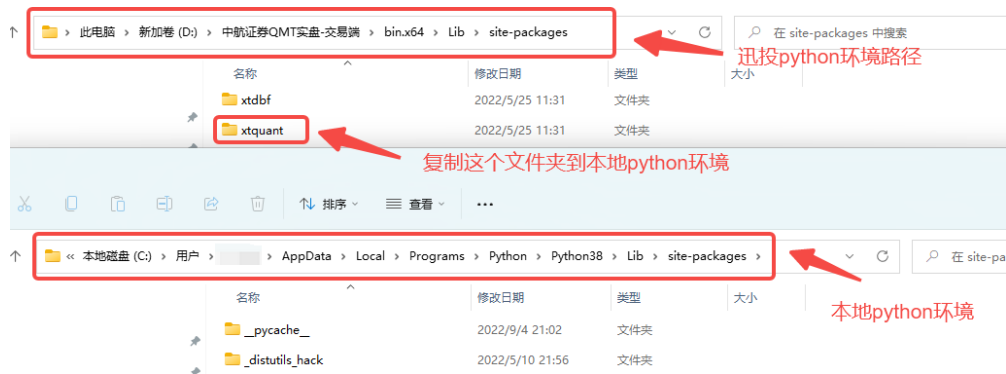


3、下载迅投 Python 库，在大 QMT 内依次点击：设置 – 模型设置 – python 库下载。（记下路径，下一步要用）



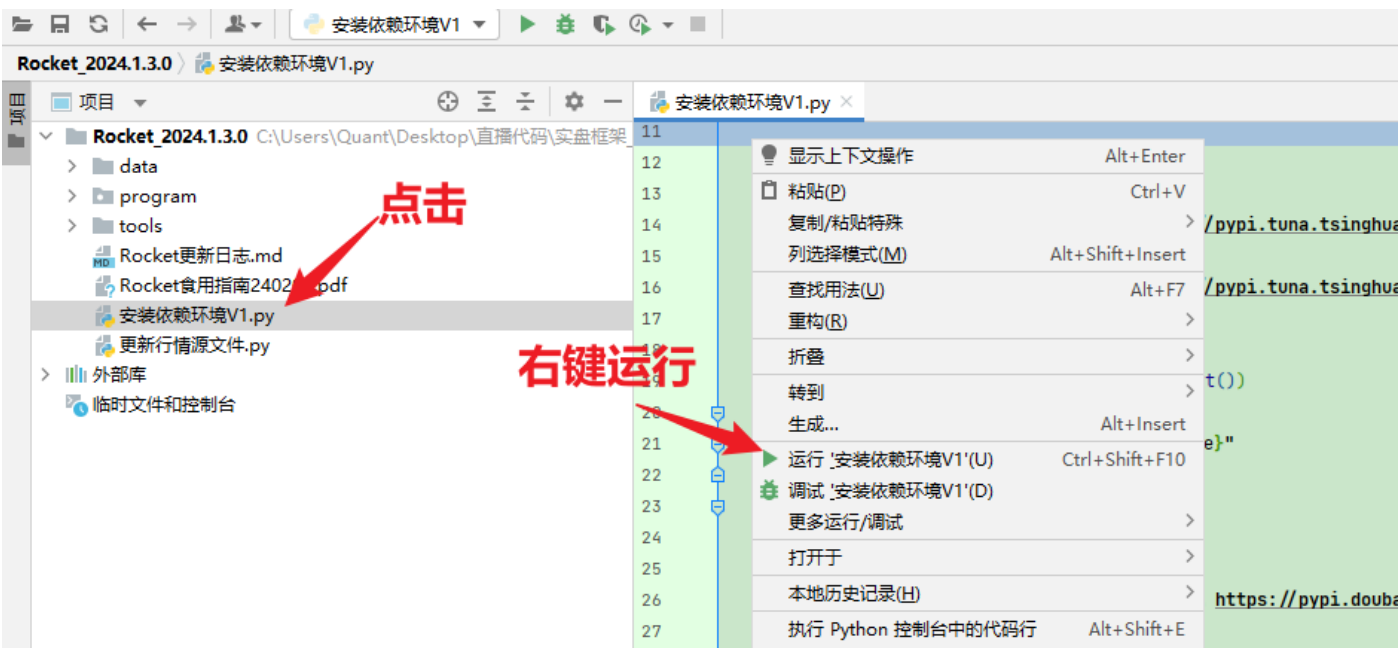
4、迅投 xtquant 包安装教程。

方案 1：手动复制迅投 Python 环境中的 xtquant 文件夹到本地的 python 环境。

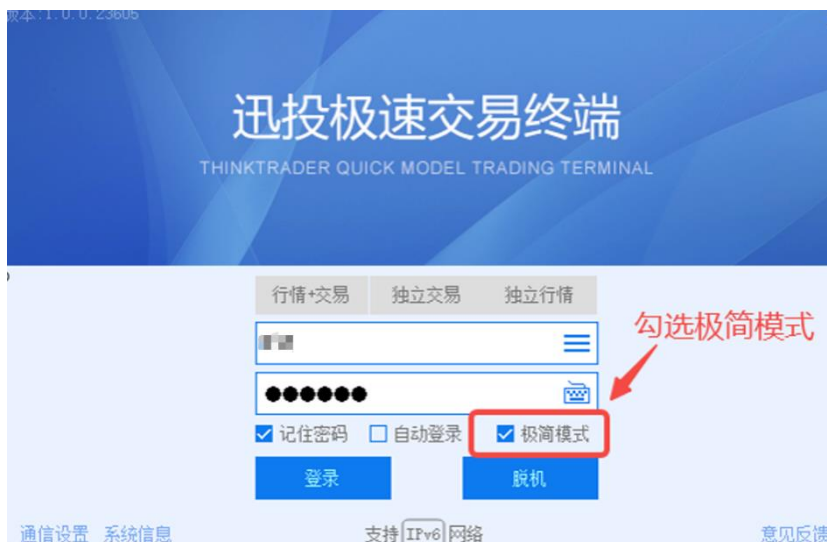


方案 2：自动安装

打开 Rocket 代码，找到《安装依赖环境.py》点击运行即可。



5、退出大 QMT，重新打开登录页面，勾选极简模式后登录。



6、实盘程序运行流程如下：

配置 config → 在交易计划.csv 中输入下单计划 → 运行程序输出交易计划 → 检查是否需要手动修改交易计划 → 确认交易计划后程序自动运行

二、 如何配置？

2.1 config 文件配置说明

1、order_exe_path： QMT 的安装位置中极简版的目录。

2、account： 券商账户号，不清楚的话可以问一下券商的客户经理哦。

3、robot_api： 企业微信机器人配置，支持配置 3 个机器人，让它们负责推送不同类别的消息

也可以三个配置都指向同一个机器人。

相关配置教程：<https://bbs.quantclass.cn/thread/10975>

4、period_offset_file： 周期和 offset 预运算数据位置

全量数据下载链接：<https://www.quantclass.cn/data/stock/stock-period-offset>

5、stg_infos： 策略配置信息。此部分配置运行《生成交易计划》项目中的

【2_生成交易计划.py】会自动生成推荐的配置，针对配置酌情修改即可。

各字段解释如下：

stg_infos.策略名: str

策略名，可为中文、英文、数字。

`stg_infos.策略名. strategy_weight`: float

策略分配的资金权重。

`stg_infos.策略名. hold_plan`: list(str)

持仓计划，通过 `period_offset.csv` 进行持仓周期查询，传入的 `str` 参数会在每日首次运行 `startup.py` 时进行校验，检查是否在 `period_offset.csv` 文件中存在，若不存在则会影响整个框架的运行。

案例，如果当前时间为 23 年 10 月 20 日，传入参数 'W_4'，周期查询为 23 年 10 月 20 日至 23 年 10 月 26 日，当日该策略的买入标的会在 10 月 26 日卖出；若传入参数 'W_3'，周期查询为 23 年 10 月 19 日至 23 年 10 月 25 日，当日该策略存在买入标的也会进行购买，会在 23 年 10 月 25 日卖出；若传入参数 'W_0'，周期查询会变成 23 年 10 月 23 日至 23 年 10 月 27 日（周期查询的逻辑时大于当日的最近一个卖出日确定周期结束日），当日购买的标的会在 23 年 10 月 27 日卖出。

`stg_infos.策略名. select_count`: int

策略的选股数（单份资金最大持股数量）

`stg_infos.策略名. stock_weight`: list

资金在股票中分配的方法，默认 `equal_weight`，表示等权分配。

本参数为 list 类型，[函数名，参数 1，参数 2，.....]，若函数无参数则无需填写参数信息。

`stg_infos.策略名. buy`: list

买入股票的方法，默认使用 `base_buy`。

本参数为 list 类型，[函数名，参数 1，参数 2，.....]，若函数无参数则无需填写参数信息。

`stg_infos.策略名. sell`: list

卖出股票的方法，默认使用 `base_sell`。

本参数为 list 类型，[函数名，参数 1，参数 2，.....]，若函数无参数则无需填写参数信息。

`stg_infos.策略名. risk`: list

风控函数，默认 `false`，表示不进行风控处理。

本参数为 list 类型, [函数名, 参数 1, 参数 2,], 若函数无参数则无需填写参数信息。

如果无需风控, 填入[false]即可。

`stg_infos.策略名. intraday_swap`: int

日内调仓标志, 可以不定义该标志, 留空等同于=0, 当该值>0 则表示该策略会进行日内调仓 (需要注意 buy 和 sell 函数的时间, 确保卖出后再买入)。

该标志=0 表示此策略不进行日内调仓, 即原尾盘卖出, 次日开盘买入的模式。

该标志=1 表示此策略进行日内调仓, 对于 hold_plan 为 str 的情况, 卖出时间计算会延后一日, 即原本尾盘卖出次日买入的情况, 变更为尾盘不卖, 次日早盘卖出后再买入。

该标志=2 表示此策略进行日内调仓, 对于 hold_plan 为 str 的情况, 买入日会比原买入日早一点, 即原本尾盘卖出次日买入的情况, 变更为尾盘卖出后, 立即进行新标的的买入。

特别注意: `stg_infos` 默认有一个【非策略选股】的策略, 请勿删除此策略的配置信息。

5、从选股策略框架中复制 `period_offset.csv` 文件至 `data` 文件夹下, 或在 `program.config.py` 中将 `period_offset_path` 修改为 `period_offset.csv` 的绝对路径

6、配置好 `rocket_plan.json` 文件后, 检查 `config` 中是否还有需要主观配置的参数, 详见代码中的注释。

2.2、交易计划.csv (允许手动修改, 允许外部程序修改)

```
交易日期,策略名称,证券代码,持仓计划,其他
2023-12-27,新策略,sh603021,5_0,山东华鹏V
2023-12-27,新策略,sh603021,5_0,山东华鹏V
```

该文件用于记录每日下单的股票信息。**交易日期**是你准备买入的日期, **策略名称**是 `stg_infos` 的键名。**持仓计划**可以不填, **其他**一列按需填写。

本文件采用追加模式, 需要新增信息在最后一行添加即可。例如你的 `Strategy_A` 准备在 22 年 4 月 11 下单 `sh600000`, 只需要在最后一行添加 `2022/4/11 ,Strategy_A ,sh600000` 即可。

目前添加的方式可以是程序自动添加, 也可以是手工添加。

2.3、当前持仓.csv （异常时允许手动修改）

策略名称	证券代码	持仓量	买入日期	计划卖出日期	订单标记	成交均价	其他
	128.SZ	160	58:49.6	2022/10/13	10103	109.87	3
	128.SZ	160	58:49.6	2022/10/13	10105	111.9074375	5
	110.SH	160	58:49.6	2022/10/13	10102	115.184125	2
	123.SZ	160	58:49.6	2022/10/13	10104	111.891875	4

该文件用于记录持仓信息（自动生成的），不同的策略买入同一只股票会被分开记录，并且它们的计划卖出日期可能是不相同的。

每次程序启动会对比本地文件和线上持仓的差异，异常数据处理规则如下：

本地信息	线上信息	处理方案
持有	不持有	删除本地的持仓信息
不持有	持有	标记为非策略选股，并将卖出日期定在 1 万日之后。
持有数量小于线上		说明被意外平掉了一部分，按照计划平仓日期由近到远依次减去。
持有数量大于线上		说明超额加仓了，记录在最远的一个计划卖出日上。

2.4、xxxx_xx_xx 买入计划.csv （异常时允许手动修改）

交易日期	策略名称	证券代码	订单标记	下单金额	是否已下单	委托编号	开盘价	成交均价	其他
2022/10/12	ent	301057.SZ	_event_20221012_0		是	302252052	16.48	16.744	
2022/10/12	_event	300281.SZ	_event_20221012_1		是	302252053	4.67	4.69	
2022/10/12	de	603535.SH	_20221012_2		是	302252041	17.24	17.759	

该文件用于记录每日下单买入股票的信息，每日第一次运行时会从交易计划.csv 中生成，之后从本地文件中读取。
历史文件会被移动到历史信息文件夹下。

2.5、xxxx_xx_xx 卖出计划.csv （异常时允许手动修改）

策略名称	证券代码	持仓量	买入日期	计划卖出日期	是否已下单	委托编号	订单标记	成交均价	其他
event	300.SZ	00	2022/10/11	2022/10/12	是	302252050	event10111	13.4	
event	600.SH	00	2022/10/11	2022/10/12	是	302252059	event10112	5.14	
event	300.SZ	00	2022/10/11	2022/10/12	是	302252060	event10112	11.48	
event	68.SH	07	2022/10/11	2022/10/12	是	302252061	event10111	28.78034797	

该文件用于记录每日下单卖出股票的信息，每日第一次运行时会从当前持仓.csv 中生成，之后从本地文件中读取。
历史文件会被移动到历史信息文件夹下。

三、 实盘常见问题及处理方案

3.1、软件是否需要一直打开？

需要的，程序是循环运行的，可以在 `program.startup` 中设置循环运行的时间间隔。

3.2、手工干预后能否重复运行？

可以的。

3.3、资金分配和计划差很多

每日下单前发现下单金额和计划下单金额存在较大差异（例如临时划走部分资金导致的），可以停止程序，打开**买入计划.csv** 手动修改下单金额的信息。

四、 可以修改或者添加的代码

4.1、 `program.utils.function.cal_allocation_of_cap`

该函数用于计算资金在各个策略中的分配，如果觉得分配的不合理，可以自行修改。

4.2、 `program.exchange.buy_method.py`

该脚本用于定义买入的方法，如果觉得 `base_buy` 不好用，可以自行添加，注意是新添加，不是删除。

4.3、 `program.exchange.sell_method.py`

该脚本用于定义卖出的方法，如果觉得 `base_sell` 不好用，可以自行添加，注意是新添加，不是删除。

4.4、 `program.exchange.risk_method.py`

该脚本用于定义风控函数，默认不做风控，如果想引入风控，又觉得 `base_risk` 不好用，可以自行添加，注意是新添加，不是删除。